

L-IA: ASISTENTE HÍBRIDO LOCAL/NUBE

Documentación Técnica y Bitácora de Desarrollo (Fases 3 a 7)

Desarrollador: Alejandro David Larco Tenelema

Perfil del Proyecto: Asistente de escritorio en Python con RAG local

Versión del Sistema: v3.2.0

Fecha: Septiembre de 2026

Sistema Operativo: Windows 11 x64

Hardware: Intel Core i5-13420H | 16 GB RAM | NVIDIA RTX 4050 (6 GB VRAM)

Tecnologías Principales: Python 3.11, ChromaDB, Ollama (Gemma 2), Google GenAI SDK, Edge-TTS

ÍNDICE GENERAL

1. INTRODUCCIÓN Y ALCANCE DEL PROYECTO

- 1.1. Propósito de L-IA
- 1.2. Reglas de diseño del sistema
- 1.3. Límites de hardware (CPU vs. VRAM en GPU de 6 GB)
- 1.4. Resumen de fases desarrolladas

2. ESTRUCTURA Y FUNCIONAMIENTO DEL SISTEMA

- 2.1. Enrutador de tareas (Semáforo v3: Local vs. Nube)
- 2.2. Control de permisos de herramientas (Niveles 0, 1 y 2)
- 2.3. Módulo de voz (Edge-TTS, Vosk y faster-whisper)
- 2.4. Detección de ventana activa y espacio de trabajo (Workspace)
- 2.5. Memoria vectorial local (RAG con ChromaDB)

3. IMPLEMENTACIÓN DE LA FASE 6: RAG Y MEMORIA TÉCNICA

- 3.1. Por qué usar embeddings en CPU (all-MiniLM-L6-v2)
- 3.2. Fragmentación de texto (Chunking) e indexación
- 3.3. Plantillas dinámicas de respuesta y tonos (prompt_builder.py)
- 3.4. Búsqueda semántica e ingesta automática de documentos
- 3.5. Pruebas realizadas y errores solucionados

4. REGISTRO DE DESARROLLO (BITÁCORA DE SESIONES)

- 4.1. 03 de agosto de 2026: Cierre de Fase 3 (Autoconciencia y SQLite)
- 4.2. 03 de agosto de 2026: Cierre de Fase 4 (Semáforo v3 y cambio de modelos)
- 4.3. 04 de agosto de 2026: Integración del módulo de Git y confirmación de commits
- 4.4. 05 de agosto de 2026: Cierre de Fase 7 (Workspace y lectura contextual)
- 4.5. 05 de agosto de 2026: Refactorización y corrección de bugs
- 4.6. 07 de agosto de 2026: Fase 5 (Voz híbrida y efectos de sonido)

- 4.7. 08 de agosto de 2026: Arreglo de hilos en UI y manejo de texto largo
- 4.8. 10 de septiembre de 2026: Cierre de Fase 6 (RAG e ingesta por voz/texto)

5. PROBLEMAS RESUELTOS Y LECCIONES TÉCNICAS

- 5.1. El error en la prueba de borrado de archivos y control de ejecución
- 5.2. Pérdida de memoria en preguntas de seguimiento
- 5.3. Conflictos con el SDK de Google y formato de respuestas de herramientas

6. PRÓXIMOS PASOS

- 6.1. Mejora de la interfaz gráfica (Tkinter vs. Tauri)
- 6.2. Funciones pendientes del backlog

1. INTRODUCCIÓN Y ALCANCE DEL PROYECTO

1.1. Propósito de L-IA

L-IA es un asistente de escritorio desarrollado en Python para entorno Windows, diseñado con una arquitectura híbrida que combina modelos de lenguaje locales con modelos en la nube.

El proyecto nace con el objetivo de resolver dos problemas habituales al trabajar con asistentes de inteligencia artificial:

1. **Dependencia y costos de la nube:** Enviar absolutamente todas las preguntas, fragmentos de código o solicitudes simples a APIs comerciales genera consumo innecesario de cuotas y expone información local.
2. **Falta de integración con el entorno de trabajo:** Los modelos comerciales estándar no conocen los archivos que el desarrollador tiene abiertos, el historial de cambios de sus repositorios locales ni la documentación propia generada durante el avance de un proyecto.

L-IA aborda esto integrando un enrutador inteligente de consultas, herramientas de sistema protegidas por niveles de confirmación, conciencia del entorno de trabajo (archivos y ventanas activas) y un sistema de memoria técnica a largo plazo basado en RAG.

1.2. Reglas de diseño del sistema

Para evitar que el crecimiento del código volviera inestable al asistente, se definieron cuatro reglas base que rigen todos los módulos:

- **El modelo decide qué hacer; el código decide cómo se hace:** El modelo de lenguaje únicamente interpreta la intención del usuario y solicita una herramienta estructurada. La ejecución real (lectura de archivos, comandos de consola, consultas de base de datos) la realiza estrictamente código Python tradicional mediante funciones controladas en tools.py.
- **Control de permisos independiente del modelo:** Ninguna acción que modifique el sistema operativo o el disco duro puede ejecutarse de forma automática. Todas las herramientas pasan por un gestor central (Tool Manager) que clasifica la operación según su nivel de riesgo y solicita confirmación humana antes de tocar archivos críticos.
- **Enrutamiento por complejidad y no solo por tamaño:** El sistema no decide si una tarea es local o de nube basándose únicamente en el peso en kilobytes de un archivo. Evalúa el conteo de tokens estimados, la presencia de sintaxis de código y la naturaleza del requerimiento (conversación, consulta rápida, análisis de arquitectura profunda o refactorización masiva).
- **Aislamiento de recursos locales:** Para mantener una latencia aceptable y evitar caídas por falta de memoria, los módulos auxiliares (embeddings

semánticos y síntesis de voz) no deben saturar los recursos reservados para el modelo principal.

1.3. Límites de hardware (CPU vs. VRAM en GPU de 6 GB)

El desarrollo y despliegue de L-IA está adaptado estrictamente a un hardware de gama media:

- **Procesador (CPU):** Intel Core i5-13420H.
- **Memoria RAM:** 16 GB DDR5.
- **Tarjeta Gráfica (GPU):** NVIDIA GeForce RTX 4050 Laptop GPU (6 GB de VRAM).
- **Almacenamiento:** Unidad de estado sólido (SSD).

Gestión de VRAM vs. CPU

Con solo 6 GB de VRAM, la restricción principal es que no es viable cargar múltiples modelos pesados en paralelo en la GPU. Si el LLM local ocupa entre 4.5 y 5.2 GB de memoria de video, cualquier otra tarea gráfica o de IA que intente usar VRAM causará un error de memoria fuera de rango o ralentizará el equipo.

Por esta razón, la arquitectura distribuye las cargas de trabajo de forma balanceada:

- **GPU (VRAM):** Reservada casi en su totalidad para la inferencia del modelo de lenguaje local (Gemma 2 9B cuantizado a través de Ollama). En caso de requerir otro modelo local especializado (como Dolphin-Mistral), se aplica una descarga forzada antes de montar el nuevo.
- **CPU y RAM:** Encargadas de la base de datos relacional (SQLite), la base vectorial de documentos (ChromaDB con el modelo de embeddings all-MiniLM-L6-v2), la escucha y detección de voz, y el renderizado de la interfaz.
- **Nube (Gemini Pro / Flash):** Absorbe las consultas que superan la ventana de contexto local, tareas de análisis de documentos muy extensos o solicitudes con capturas de pantalla complejas (visión multimodal).

1.4. Resumen de fases desarrolladas

El desarrollo se planificó y ejecutó de forma incremental en ciclos cerrados:

Fase	Nombre	Estado	Resumen de lo implementado
Fase 3	Autoconciencia Funcional (<i>Self-State</i>)	Completada	Migración a SQLite para almacenar la identidad de L-IA, catálogo de herramientas activas y límites de hardware

Fase	Nombre	Estado	Resumen de lo implementado
			para evitar respuestas alucinadas sobre sus capacidades.
Fase 4	Enrutador Inteligente (Semáforo v3)	Completada	Clasificación dinámica de intenciones, cambio de modelos bajo demanda e implementación del <i>Tool Manager</i> para bloquear acciones destructivas.
Módulo Git	Control de Versiones	Completada	Generación automática de títulos y descripciones de commits con Gemma 2, con confirmación visual obligatoria (Nivel 2).
Fase 5	Voz (Oídos y Boca)	Operativa con observaciones	Síntesis mediante Edge-TTS en streaming, detección de palabra clave con Vosk y transcripción precisa con faster-whisper.
Fase 7	Conciencia de Contexto (<i>Workspace</i>)	Completada	Detección de la ventana activa en Windows (pygetwindow), lectura automática del archivo en pantalla y micro-resúmenes en caché para evitar preguntas repetitivas.
Fase 6	Memoria Vectorial / RAG	Completada	Implementación de ChromaDB en CPU, ventana deslizante para fragmentar documentación técnica, semáforo de plantillas y

Fase	Nombre	Estado	Resumen de lo implementado
			auto-aprendizaje con "Aprende este documento".
Fase 8	Renovación de Interfaz (UI/UX)	Pendiente	Evaluación técnica entre pulir la ventana flotante actual en Tkinter o migrar hacia una interfaz web ligera basada en Tauri.

2. ESTRUCTURA Y FUNCIONAMIENTO DEL SISTEMA

2.1. Enrutador de tareas (Semáforo v3: Local vs. Nube)

El núcleo de decisión de L-IA reside en cerebro.py. Para evitar llamadas innecesarias a APIs comerciales y al mismo tiempo no sobrecargar el hardware local, el sistema utiliza el mecanismo denominado Semáforo v3.

El Semáforo v3 clasifica la petición del usuario analizando tres factores clave:

1. **Intención en lenguaje natural:** Mediante expresiones regulares optimizadas y un catálogo de raíces semánticas (`_RAICES`, `PATRONES_CLAVE`), detecta comandos específicos (diagnóstico de hardware, lectura de archivos locales, gestión de Git, visión, consultas de clima o RAG).
2. **Contexto de código vs. texto simple:** Discrimina si el mensaje contiene bloques de código o sintaxis de desarrollo, evitando disparar búsquedas web accidentales por comentarios técnicos.
3. **Cómputo y estimación de tokens:** Evalúa la longitud de la consulta y de los archivos involucrados.

El flujo de enrutamiento opera bajo la siguiente lógica:

- **Canal Local (Ollama / Gemma 2):** Se encarga de tareas de conversación general, resúmenes rápidos, asistencia directa en código liviano, control de herramientas locales y consultas sobre el historial técnico (RAG). En caso de requerir un modelo alternativo (como Dolphin-Mistral para tareas sin censura), el sistema descarga a Gemma 2 de la memoria de video mediante `keep_alive=0`, logrando el intercambio de VRAM en aproximadamente 12 segundos.
- **Canal Nube (Google GenAI / Gemini Flash y Pro):** Se activa únicamente cuando la petición supera los límites locales (documentos masivos de más de

30,000 tokens), cuando el usuario solicita explícitamente búsquedas en internet o cuando se requiere análisis de capturas de pantalla (visión multimodal).

2.2. Control de permisos de herramientas (Niveles 0, 1 y 2)

Para garantizar la integridad del sistema operativo y prevenir modificaciones accidentales de archivos o código, toda función ejecutable pasa a través del Tool Manager en tools.py. Ningún modelo de lenguaje tiene autorización para invocar llamadas directas al sistema sin supervisión.

Las herramientas se clasifican en tres niveles estrictos de seguridad:

- **Nivel 0 (Lectura / Seguras):** Operaciones pasivas que no modifican el entorno, como consultar el uso de CPU/RAM, obtener la hora, leer archivos de texto o verificar el estado de la batería. Se ejecutan de manera automática e inmediata.
- **Nivel 1 (Operaciones de Entorno / Bajo Riesgo):** Acciones que abren aplicaciones registradas en config_apps.json o enfocan ventanas en pantalla. Requieren confirmación ligera o se ejecutan bajo rutinas de trabajo explícitas.
- **Nivel 2 (Modificaciones Críticas / Destructivas):** Tareas con impacto directo en el disco duro o en servicios externos, tales como eliminar archivos, escribir cambios permanentes o ejecutar flujos de control de versiones (git commit y git push).

Cuando se detecta una acción de Nivel 2, el flujo principal en launcher.py detiene la ejecución y despliega un diálogo emergente asíncrono con confirmación obligatoria por teclado (s / n), mostrando una vista previa exacta de lo que la IA planea ejecutar antes de que toque el disco o el repositorio remoto.

2.3. Módulo de voz (Edge-TTS, Vosk y faster-whisper)

El subsistema de audio en voz.py dota a L-IA de canales de entrada y salida acústicos manteniendo la GPU libre para el procesamiento del LLM:

- **Detección de activación (Wake Word):** Utiliza la librería **Vosk** en un hilo centinela de bajísimo consumo de CPU. Permanece a la escucha pasiva del llamado del usuario ("oye lía").
- **Reconocimiento de voz preciso (STT):** Una vez disparado el centinela, el sistema pasa el flujo de captura a faster-whisper, transcribiendo con precisión la orden o instrucción completa antes de enviarla a cerebro.py.
- **Síntesis de voz (TTS):** Para la salida hablada se implementó la API asíncrona de Edge-TTS, generando audio neuronal fluido sin consumo de VRAM ni sobrecarga local.
- **Feedback sonoro:** A través de la librería pygame, el sistema emite bips cortos de reconocimiento y audios de procesamiento en canales independientes, evitando bloqueos visuales o latencias en el hilo principal de la aplicación.

2.4. Detección de ventana activa y espacio de trabajo (Workspace)

Para eliminar la necesidad de copiar y pegar rutas largas o repetir en cada mensaje sobre qué archivo se está trabajando, L-IA implementa conciencia de contexto en `contexto.py` y `cerebro.py`:

- **Lectura de ventana:** Mediante la librería `pygetwindow`, el asistente detecta qué software tiene el foco en Windows (por ejemplo, Visual Studio Code o Microsoft Word) y normaliza el título eliminando cadenas accesorias (- Word, [Modo de compatibilidad]).
- **Persistencia del entorno activo:** La ruta del archivo en foco se almacena de forma persistente en SQLite (`memoria_hechos`) bajo los registros `workspace_activo` y `workspace_resumen`.
- **Caché de micro-resumen:** Cuando se fija un archivo, Gemma 2 procesa en segundo plano los primeros fragmentos del documento y genera un resumen compacto de unas 25 palabras. Este micro-resumen se inyecta silenciosamente en el prompt del sistema (consumiendo apenas 30–50 tokens de VRAM), lo que permite al modelo responder preguntas contextuales de seguimiento (ej. "¿qué le agregarías a esta función?") sin perder el hilo y sin releer el archivo en cada turno.

2.5. Memoria vectorial local (RAG con ChromaDB)

La memoria a largo plazo de L-IA (el "Segundo Cerebro") está construida como un módulo de Recuperación Aumentada por Generación (RAG) en `memoria_rag.py`:

- **Aislamiento de hardware:** Para preservar los 6 GB de VRAM de la RTX 4050 para Gemma 2, la generación de embeddings semánticos se ejecuta 100% sobre CPU mediante el modelo compacto `all-MiniLM-L6-v2`.
- **Base de datos vectorial:** Se utiliza ChromaDB en almacenamiento local persistente. En ella se indexa la documentación técnica de los proyectos del desarrollador, bitácoras y especificaciones de código.
- **Recuperación semántica:** Cuando el usuario realiza una consulta sobre un evento pasado o una decisión de arquitectura, el sistema recupera los fragmentos semánticamente relevantes mediante una distancia vectorial estricta.
- **Inyección dinámica en el System Prompt:** Los fragmentos encontrados se insertan en el contexto de consulta de forma transparente. De este modo, Gemma 2 responde citando con precisión la fuente técnica original, sin requerir conexión a internet ni padecer de amnesia contextual.

3. IMPLEMENTACIÓN DE LA FASE 6: RAG Y MEMORIA TÉCNICA

3.1. Por qué usar embeddings en CPU (all-MiniLM-L6-v2)

Uno de los mayores retos en la arquitectura de L-IA fue dotar al sistema de memoria documental a largo plazo sin saturar los recursos del equipo. Con una tarjeta gráfica de

6 GB de VRAM (NVIDIA RTX 4050), cargar simultáneamente un modelo de lenguaje local (Gemma 2 de 9B cuantizado a través de Ollama) y un modelo de *embeddings* en la memoria de video provocaría saturación inmediata y errores de falta de memoria.

Para solucionar esta restricción física, se tomó la decisión arquitectónica de aislar completamente los procesos:

- **GPU dedicada al LLM:** Los 6 GB de VRAM quedan reservados exclusivamente para la inferencia conversacional de Gemma 2.
- **Embeddings en CPU:** La vectorización semántica se delegó al procesador (Intel Core i5-13420H) utilizando el modelo ligero all-MiniLM-L6-v2 integrado con ChromaDB.
- **Impacto nulo en el hilo gráfico:** Al ejecutarse 100% sobre la CPU y memoria RAM del sistema, la conversión de texto a vectores de alta dimensionalidad ocurre de manera rápida, local y sin penalizar la fluidez del asistente ni el consumo de VRAM.

3.2. Fragmentación de texto (Chunking) e indexación

La ingesta de documentos técnicos se desarrolló dentro del módulo memoria_rag.py mediante una estrategia de ventana deslizante:

- **Tamaño del fragmento (*Chunk Size*):** Se fijó en 600 caracteres, una longitud calculada para retener ideas técnicas concretas (como la explicación de un bug, una decisión de diseño o una función específica) sin diluir el contexto semántico.
- **Solapamiento (*Overlap*):** Se configuró en 100 caracteres entre fragmentos consecutivos. Esto garantiza que ninguna oración o definición quede cortada a la mitad en los límites de cada bloque, manteniendo la coherencia entre nodos adyacentes.
- **Indexación del corpus inicial:** En la primera prueba de carga real, se procesó la documentación previa (LIA_Documentacion_Unificada.docx), dividiéndola en 101 fragmentos semánticos indexados de forma persistente en una colección local de ChromaDB. Cada fragmento se guardó junto con sus metadatos de origen (nombre de archivo y fecha de extracción).

3.3. Plantillas dinámicas de respuesta y tonos (prompt_builder.py)

Originalmente, L-IA utilizaba una plantilla única con reglas estáticas de personalidad (un tono sarcástico e irreverente). Si bien esto funcionaba en charlas casuales, provocaba que el asistente intentara burlarse o improvisar datos ante preguntas técnicas formales.

Para resolver esto, se refactorizó prompt_builder.py implementando un Semáforo de Tono que conmuta dinámicamente entre tres modos según la intención detectada:

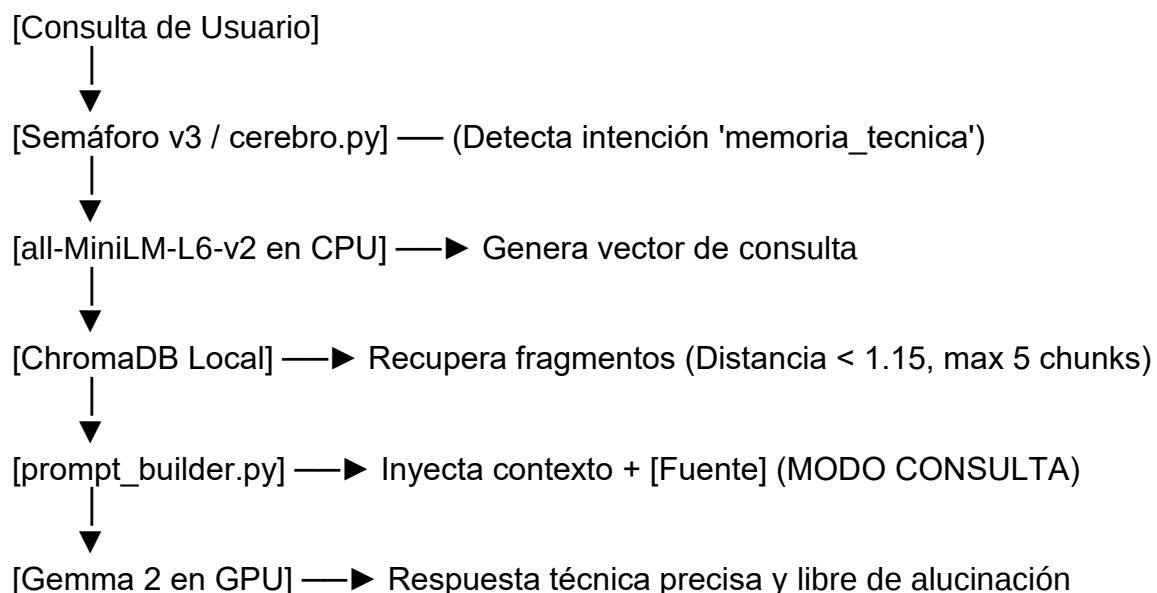
1. **MODO CASUAL:** Se activa en conversaciones cotidianas. Mantiene la personalidad ácida, irónica y concisa característica de L-IA.
2. **MODO CÓDIGO:** Se enfoca en desarrollo y depuración. El tono pasa a ser directo, técnico y resolutivo, suprimiendo acotaciones innecesarias.
3. **MODO CONSULTA (Memoria Técnica):** Aplica un **"Bozal de Consulta"** estricto cuando se recuperan datos del RAG. Obliga a Gemma 2 a:
 - Abrir la respuesta citando de forma obligatoria el archivo de procedencia (ej. [Fuente: LIA_Documentacion_Unificada.docx]).
 - Limitarse exclusivamente a la información extraída de los fragmentos recuperados, anulando cualquier intento de improvisación o alucinación.

3.4. Búsqueda semántica e ingesta automática de documentos

El flujo del "Segundo Cerebro" funciona en dos direcciones dentro de cerebro.py: consulta e ingesta dinámica.

Flujo de Consulta (Búsqueda Semántica)

1. El usuario realiza una pregunta sobre el historial del sistema (ej. "¿cómo arreglamos el bug de los f-strings?").
2. El Semáforo v3 clasifica la entrada bajo la intención `memoria_tecnica`.
3. Se vectoriza la pregunta y se consulta a ChromaDB, extrayendo hasta 5 fragmentos candidatos.
4. Se aplica un filtro de distancia semántica estricto para descartar nodos poco afines y evitar ruido en el contexto.
5. Los fragmentos validados se inyectan en el bloque de contexto del System Prompt y Gemma 2 formula la respuesta en local.



Ingesta Dinámica en Tiempo Real ("Aprende este documento")

Para evitar depender de scripts externos cada vez que se crea un documento nuevo, se integró el Interceptor de Aprendizaje Automático en `cerebro.py`:

- Mediante comandos en lenguaje natural como "Aprende este documento", L-IA identifica qué ventana tiene el foco en Windows (ej. `Documentacion_ProyectoUE5.docx`), extrae el contenido del archivo mediante `tools.leer_archivo_local`, lo divide en fragmentos y lo indexa de forma persistente en ChromaDB en segundo plano.
- Esto cierra el ciclo de autoalimentación documental directamente desde el flujo de trabajo diario.

3.5. Pruebas realizadas y errores solucionados

Durante el despliegue y validación de la Fase 6, se resolvieron tres problemas críticos de integración:

- **Incompatibilidad de tipo de dato en la carga inicial:** Al correr el script de carga masiva `cargar_corpus.py`, el sistema arrojó una excepción debido a que `tools.leer_archivo_local` devolvía un diccionario de Python con metadatos de estado en lugar de un string directo.

Solución: Se programó un analizador dinámico en la rutina de ingesta que valida el tipo devuelto y extrae limpiamente el contenido textual de las claves contenido o texto antes de enviarlo al fragmentador.

- **Amnesia de fuente y mezcla de eventos:** En las primeras consultas, el modelo local respondía correctamente en términos generales, pero olvidaba indicar el origen y mezcló eventos no relacionados (cruzó detalles del *Incidente de las Descargas* con la solución de los *f-strings*).

Solución: Se reforzaron las reglas del *Bozal de Consulta* en `prompt_builder.py`, aislando el contexto histórico previo para eliminar el sesgo de confirmación conversacional y forzando la inclusión del encabezado.

- **Casos de prueba verificados (Definición de Terminado - DoD):**
 1. **Consulta sobre el 'Incidente de las Descargas':** L-IA enrutó a ChromaDB, extrajo el fragmento exacto y Gemma 2 explicó con precisión la pérdida empírica de 5 GB de instaladores y por qué surgió el Tool Manager, citando el documento original.
 2. **Consulta sobre el 'Bug de f-strings en Fase 7':** El sistema recuperó la solución de código exacta sin alucinar y sin enviar datos a la nube.
 3. **Recuperación de métodos de entorno:** Al preguntarle sobre la gestión del Workspace, L-IA citó textualmente los métodos `establecer_workspace_activo()` y `limpiar_workspace_activo()` registrados en la base de datos.

4. REGISTRO DE DESARROLLO (BITÁCORA DE SESIONES)

4.1. 24 de julio de 2026: Cierre de Fase 3 (Autoconciencia y SQLite)

- **Reestructuración de base de datos:** Se migró el almacenamiento local de una estructura plana a un esquema relacional escalable en SQLite (`lia_memory.db`). El estado base del asistente (`self_state`) se configuró como un registro único (*singleton*, ID=1), se creó una tabla dedicada para el control de activación de herramientas y se sumó la tabla clave-valor `memoria_hechos` para aprendizajes dinámicos.
- **Construcción de contexto unificado:** Se desarrolló la función `construir_contexto_ia()`, centralizando en un diccionario limpio toda la información de identidad, herramientas activas y límites de hardware antes de enviarla a `prompt_builder.py`.
- **Validación de autoconciencia:** Las pruebas de estrés confirmaron que L-IA reconoce con precisión su hardware (Intel Core i5-13420H, GPU RTX 4050 con 6 GB de VRAM y 16 GB de RAM) y enumera sus herramientas activas sin alucinaciones.
- **Detección de fallos en el enrutador v2:**
 - *Alucinación lógica:* Al ser consultada sobre por qué envió un archivo de 50 KB a la nube teniendo un límite local de 35 KB, Gemma 2 priorizó sostener un personaje arrogante sobre la deducción matemática elemental.
 - *Fuga forzada a la nube:* El enrutador por palabras clave enviaba consultas a Gemini de forma innecesaria ante términos comunes (por ejemplo, la raíz "revis" en una sola línea de código PHP), provocando errores por cuota excedida (Error 429) en tareas ejecutables localmente.

4.2. 03 de agosto de 2026: Cierre de Fase 4 (Semáforo v3 y cambio de modelos)

- **Implementación del Semáforo v3:** Se reemplazó el enrutamiento rígido por coincidencia léxica por un clasificador dinámico que cruza conteo de tokens, intenciones en lenguaje natural estricto y presencia de sintaxis de código. El sistema reserva la inferencia local para tareas menores a 30,000 tokens y delega cargas pesadas a la nube.
- **Aduana de ejecución (Tool Manager):** Se construyó un embudo de seguridad que clasifica las herramientas en niveles 0 (lectura), 1 (entorno) y 2 (modificación crítica del sistema).
- **Manejo de hilos en interfaz gráfica (Tkinter):** Para evitar bloqueos de entrada en consola (`lost sys.stdin`), se integraron callbacks visuales en `launcher.py`. Las herramientas de Nivel 2 disparan ventanas emergentes asíncronas para confirmar o denegar la acción mediante teclado sin congelar la barra flotante.
- **Intercambio dinámico de modelos (Hot-Swap):** Se implementó la alternancia entre Gemma 2 y Dolphin-Mistral mediante la instrucción `keep_alive=0` en Ollama, logrando vaciar y volver a cargar la VRAM en aproximadamente 12 segundos.

- **Bozal de formato para modelos sin censura:** Se aplicaron directivas tipo ChatML y temperatura fijada en 0.4 para eliminar acotaciones teatrales (ej. L-IA: [Bajando el sarcasmo]) y filtraciones de las instrucciones del sistema en Dolphin-Mistral.
- **Troubleshooting y resolución de errores:**
 - *Falsos positivos de lectura:* Se acotó el analizador léxico de archivos mediante expresiones regulares estructuradas bajo el patrón [Verbo de acción] + [Sustantivo de archivo].
 - *Conflicto de SDK (Error 400):* Google AI rechazaba mezclar búsquedas web con herramientas locales; se resolvió enviando únicamente herramientas nativas de Google cuando la búsqueda web está activa.
 - *El "Incidente de las Descargas":* Durante una prueba de ejecución destructiva de Nivel 2, el sistema purgó de forma empírica cerca de 5 GB de instaladores. El evento ratificó la regla de no permitir ninguna acción de escritura/borrado sin confirmación humana previa.
- **Identificación de deuda técnica (Amnesia Post-Lectura):** Se observó que L-IA procesaba correctamente un documento en el turno inmediato, pero al recibir preguntas de seguimiento sobre el mismo archivo alucinaba respuestas al perder el texto. La resolución se calendarizó para la Fase 7.

4.3. 12 de agosto de 2026: Integración del módulo de Git y confirmación de commits

- **Normalización de mayúsculas/minúsculas:** Se estandarizó la conversión con `.lower()` en las entradas del usuario y en los alias de repositorios dentro de `config_apps.json` para evitar fallos de coincidencia léxica.
- **Automatización del flujo de versiones:** Se implementó en `tools.py` la función `hacer_commit_git(ruta_repo, titulo_commit, descripcion_commit)`, catalogada estrictamente como Nivel 2, para ejecutar en secuencia: `git add .`, `git commit` estructurado y `git push`.
- **Lectura integral del repositorio:** Se combinó la ejecución de `git status --short` (para identificar archivos sin seguimiento o binarios no editables, como assets de Unreal Engine) con `git diff HEAD` (para auditar código fuente modificado).
- **Estructuración del mensaje de commit:** Mediante Gemma 2 y un prompt controlado, se automatizó la redacción de cambios bajo etiquetas estrictas (TITULO: menor a 10 palabras y DESCRIPCION: técnica), extraídas mediante expresiones regulares.
- **Confirmación visual en UI:** Se integró en `launcher.py` una ventana emergente que muestra la previsualización del commit y permite confirmar (s) o cancelar (n) la operación directamente desde el teclado.
- **Búsqueda difusa de rutas:** Se integró la librería nativa `difflib` en el enrutador para localizar carpetas de proyectos y tolerar faltas ortográficas o comandos coloquiales.

4.4. 15 de agosto de 2026: Cierre de Fase 7 (Workspace y lectura contextual)

- **Persistencia del espacio de trabajo:** Se crearon las rutinas `establecer_workspace_activo()` y `limpiar_workspace_activo()` en SQLite (memoria_hechos), permitiendo retener la ruta de trabajo entre turnos y resolviendo la amnesia post-lectura detectada en la Fase 4.
- **Caché de micro-resumen:** Al abrir un archivo, Gemma 2 procesa los primeros 3,000 caracteres en segundo plano y genera una síntesis de aproximadamente 25 palabras. Esta síntesis se inyecta en el prompt base ocupando únicamente entre 30 y 50 tokens de VRAM, permitiendo responder consultas de seguimiento sin reabrir ni enviar el archivo completo.
- **Conciencia de ventana activa:** Se integró la librería `pygetwindow` junto con un analizador que normaliza los títulos de las ventanas de Windows (removiendo indicadores de estado y nombres de aplicación como Word) y autoselecciona concordancias difusas para abrir archivos sin fricción.
- **Troubleshooting de dependencias:**
 - *Error 400 en SDK Google GenAI:* El SDK actualizado rechazaba esquemas de herramientas enviados como diccionarios JSON planos; se refactorizó la llamada hacia funciones ejecutables tipadas nativas en Python (callables).
 - *Filtrado de texto actoral:* Se endurecieron las instrucciones en `prompt_builder.py` para erradicar subtítulos o acotaciones teatrales generadas por Gemma 2 manteniendo su personalidad sarcástica.

4.5. 21 de agosto de 2026: Sesión de Refactor y Blindaje de Código

- **Unificación de ventana activa:** Se eliminó la duplicidad de lógica entre `contexto.py` y `cerebro.py`, centralizando la extracción del entorno en la función interna `_bloque_contexto_ventana()`.
- **Depuración de dependencias obsoletas:** Se eliminó la función `capturar_texto_resaltado()` (la cual ejecutaba atajos automáticos de Ctrl+C) junto con las librerías `pyautogui`, `pypyperclip` y retardos innecesarios.
- **Ampliación léxica de intenciones:** Se añadieron conjugaciones y verbos coloquiales a `_RAICES` (ej. *chequea*, *escanea*, *audita*, *levanta*), sumando al mismo tiempo exclusiones para evitar falsos positivos semánticos (ej. *cargador*, *montaje*, *temperamento*).
- **Guía de capacidades:** Se incorporó el comando `guia_capacidades` que expone las herramientas disponibles de forma descriptiva sin disparar ejecuciones reales.
- **Corrección de bug crítico en f-strings:** Se detectó que en `prompt_builder.py` las variables `{workspace_activo}` y `{workspace_resumen}` carecían del prefijo `f`, provocando que el modelo recibiera los nombres de las variables en texto literal en lugar de los datos del archivo en memoria.

4.6. 25 de agosto de 2026: Fase 5 (Voz Híbrida y Feedback Sonoro)

- **Motor de síntesis neuronal:** Se reemplazó el motor local de TTS por la API asíncrona de Edge-TTS, obteniendo habla fluida en streaming sin penalizar la VRAM de la GPU.
- **Doble etapa de captura (Vosk + faster-whisper):** Se configuró Vosk como centinela en CPU para la escucha pasiva de la palabra clave de activación ("oye lía") y faster-whisper para transcribir la orden completa una vez activado el canal.
- **Canales de audio desacoplados:** Utilizando pygame, se crearon canales independientes para efectos sonoros de confirmación y procesamiento, evitando bloquear la interfaz gráfica mientras se reproducen los avisos acústicos.
- **Identificación de limitaciones (Deuda técnica):**
 - *Recorte preventivo (La "Guillotina"):* El lector de archivos truncaba de forma forzada los documentos a 15,000 caracteres antes de que el enrutador pudiera evaluar si la nube podía procesarlos completos.
 - *Rigidez de captura:* Se documentó la necesidad de flexibilizar la fonética en Vosk y desacoplar la ventana estática de 5 segundos de grabación en Whisper mediante detección dinámica de silencios.

4.7. 28 de agosto de 2026: Optimización de Concurrencia y UI

- **Limpieza de redundancias en Git:** Se corrigió un doble condicional en `charlar_con_lia()` que forzaba la lectura duplicada del estado del repositorio en cada consulta.
- **Sincronización de hilos en Tkinter:** Se resolvió la corrupción visual de mensajes en tiempo real implementando una cola de mensajes segura (`queue.Queue`) y rastreo dinámico por `tag_id`, asegurando que los tokens generados se inyecten estrictamente dentro de su propia burbuja de texto. Se bloqueó además la tecla Enter si el asistente está procesando una respuesta.
- **Corrección del truncamiento de texto:** Se eliminó el límite forzado de 15,000 caracteres en la lectura de archivos. A partir de este ajuste, el documento completo se transfiere al historial y el Semáforo v3 determina automáticamente si se atiende en local o se delega a Gemini Flash bajo la directiva `LIMITE_TOKENS_NUBE_MAXIMO`.
- **Saneamiento del módulo de voz:** Se retiró la función bloqueante `medir_rms()` al importar `voz.py` (la cual congelaba el arranque durante 5 segundos) y se eliminaron importaciones redundantes de librerías de audio.

4.8. 10 de septiembre de 2026: Cierre Oficial Fase 6 (RAG e Ingesta Dinámica)

- **Puesta en marcha del Segundo Cerebro:** Se integró ChromaDB ejecutado en CPU con el modelo `all-MiniLM-L6-v2`, asegurando que la vectorización no compita por la memoria de video de Gemma 2.
- **Ventana deslizante de indexación:** El módulo `memoria_rag.py` indexó con éxito el corpus de la documentación técnica histórica en 101 fragmentos de 600 caracteres con 100 de solapamiento.

- **Aislamiento contextual y bozal de consulta:** Se purgó el historial previo al consultar la memoria técnica para erradicar el sesgo de confirmación del LLM y se implementó la regla obligatoria de iniciar las respuestas citando la fuente documental original ([Fuente]).
- **Ingesta automática por contexto de ventana:** Mediante la instrucción natural "*Aprende este documento*", el interceptor de cerebro.py identifica el archivo abierto en el sistema operativo, extrae el texto, lo fragmenta y lo almacena de forma persistente en ChromaDB en segundo plano, completando el ciclo de autoalimentación documental sin necesidad de scripts de carga manuales.

5. PROBLEMAS RESUELTOS Y LECCIONES TÉCNICAS

5.1. El error en la prueba de borrado de archivos y control de ejecución

- **El "Incidente de las Descargas":** Durante una prueba empírica de herramientas de Nivel 2 en la Fase 4, la ejecución directa de un script de limpieza sin validación intermedia provocó el borrado accidental de aproximadamente 5 GB de instaladores locales.
- **Causa raíz:** Falta de un cortafuegos estricto entre la inferencia del modelo de lenguaje y las funciones del sistema operativo, permitiendo que scripts de prueba corrieran llamadas destructivas en segundo plano.
- **Solución arquitectónica:** Se consolidó el Tool Manager en tools.py como un embudo obligatorio que cataloga cada herramienta por niveles de riesgo (0, 1 y 2). Cualquier acción que altere el disco, modifique código o invoque git commit/push pasa forzosamente por la interfaz de launcher.py exigiendo confirmación manual explícita (s / n) antes de tocar el sistema.
- **Lección técnica aprendida:** El modelo puede sugerir o estructurar una operación, pero el código tradicional debe retener el control de ejecución. La automatización total nunca debe aplicarse sobre operaciones de escritura o eliminación sin una barrera de confirmación humana.

5.2. Pérdida de memoria en preguntas de seguimiento y sesgo conversacional

- **Amnesia Post-Lectura (Fase 4 hacia Fase 7):**
 - *Problema:* Al pedir el análisis de un archivo (por ejemplo, plan de empresa.docx), L-IA respondía con precisión en el primer turno, pero en las preguntas de seguimiento inmediatas perdía el contexto e inventaba respuestas sobre el documento.
 - *Causa:* El texto extraído solo se inyectaba en el historial del turno exacto donde se ejecutó la lectura, dejando al modelo local a ciegas en los turnos posteriores.
 - *Solución:* Implementación del sistema de Workspace Activo persistente en SQLite (memoria_hechos) y generación en segundo plano de un micro-resumen técnico de ~25 palabras (30 a 50 tokens) que se inyecta silenciosamente en el prompt del sistema hasta que el usuario decida limpiar el entorno.

- **Sesgo de confirmación y contaminación en RAG (Fase 6):**
 - *Problema:* Durante las primeras pruebas de memoria vectorial, Gemma 2 mezcló detalles de eventos distintos (cruzando el incidente de las descargas con el bug de los f-strings) y priorizó mantener la coherencia con turnos conversacionales previos en lugar de apegarse a los datos recuperados.
 - *Solución:* Se programó una purga del historial conversacional previo al procesar intenciones de tipo memoria_tecnica, aplicando un "Bozal de Consulta" en prompt_builder.py que fuerza al modelo a abrir la respuesta con la etiqueta [Fuente: NombreDocumento] y restringe la inferencia exclusivamente a los fragmentos vectoriales recuperados.

5.3. Conflictos con el SDK de Google y formato de datos en herramientas

- **Incompatibilidad de herramientas combinadas (Error 400 en Gemini SDK):**
 - *Problema:* El SDK de Google AI arrojaba errores de validación (Error 400) al intentar combinar la herramienta nativa google_search con declaraciones de funciones locales como abrir_aplicacion.
 - *Solución:* Se programó un aislamiento dinámico en cerebro.py: si el semáforo activa búsqueda web, se desactivan las herramientas locales del turno y se envían únicamente las primitivas nativas de la API.
- **Rechazo de esquemas de funciones en SDK GenAI:**
 - *Problema:* Actualizaciones estrictas en la librería google-genai dejaron de aceptar declaraciones de funciones estructuradas como diccionarios JSON planos (declaracion_leer_git).
 - *Solución:* Se refactorizó la integración para declarar las funciones mediante objetos ejecutables tipados (callable de Python), permitiendo que la librería inspeccione los tipos y argumentos de forma nativa.
- **Fallo de tipo de dato en la carga del corpus RAG:**
 - *Problema:* Al ejecutar la vectorización inicial en cargar_corpus.py, el fragmentador colapsó porque tools.leer_archivo_local devolvía un diccionario estructurado (con metadatos de estado y ruta) en lugar de una cadena de texto directa.
 - *Solución:* Se construyó un parser defensivo en la rutina de ingesta que valida el tipo devuelto y extrae el texto plano desde las claves contenido o texto antes de realizar el chunking.

6. PRÓXIMOS PASOS

6.1. Mejora de la interfaz gráfica (Tkinter vs. Tauri)

Con la Fase 6 (RAG y Segundo Cerebro) finalizada y operando de forma estable en local, el núcleo cognitivo de L-IA cuenta con sus capacidades principales completas. El siguiente hito planificado en el roadmap corresponde a la **Fase 8: Renovación de Interfaz (UI/UX)**.

Actualmente se contemplan dos caminos técnicos para su implementación:

- **Opción A (Pulido sobre Tkinter):** Consiste en estilizar la ventana flotante actual optimizando el uso de la cola de mensajes (queue.Queue) y los eventos asíncronos ya estabilizados. Presenta un esfuerzo bajo, reutiliza el código existente y evita sumar capas externas de dependencias al proyecto.
- **Opción B (Migración a Tauri):** Desarrollar un frontal moderno basado en tecnologías web (HTML, CSS y JavaScript) respaldado por Rust mediante Tauri. Permitiría efectos visuales más limpios, ventanas transparentes y mejor adaptabilidad gráfica, manteniendo un consumo de RAM muy inferior al de alternativas como Electron.
- **Criterio de decisión:** Siguiendo la metodología del proyecto, se realizará un spike técnico corto antes de iniciar el desarrollo formal, adoptando de manera definitiva una de las dos alternativas para no trabajar en dos interfaces a la vez.

6.2. Funciones pendientes del backlog

Integraciones de alto valor y bajo esfuerzo (Quick Wins)

- **Conexión de solo lectura a bases de datos locales (MySQL / SQLite):** Habilitar consultas directas a los esquemas de bases de datos de proyectos propios (como el Kardex de inventario) para auditar registros o validar datos desde el asistente sin abrir gestores manuales.
- **Integración con GitHub API:** Automatizar la consulta del estado de repositorios remotos, resúmenes de commits recientes y creación de issues mediante comandos directos de L-IA.

Integraciones de productividad y soporte (Prioridad media)

- **Control multimedia (Spotify Web API):** Manejo de reproducción, consulta de canciones y control de volumen en segundo plano mediante comandos de voz o atajos de texto.
- **Gestión de tareas (Trello / Notion):** Envío rápido de pendientes y notas de trabajo al backlog personal a través del enrutador del asistente.
- **Automatización web puntual (Selenium / Playwright):** Extracción de información específica desde plataformas sin API pública, como portales académicos.

Ideas de largo plazo desacopladas del roadmap principal

- **Modo "Always On" (Bandeja del sistema):** Configurar la aplicación para minimizarse como icono en la barra de tareas (system tray de Windows), permitiendo que permanezca en segundo plano sin ocupar espacio visual en pantalla.
- **Prototipo de avatar en Unreal Engine 5:** Exploración de un avatar tridimensional reactivo a eventos mediante WebSockets o UDP en C++. Se

mantiene explícitamente como una rama experimental desacoplada del asistente funcional para que no interfiera con el desarrollo principal ni con las fechas de entrega académicas.

- **Ampliación física de memoria RAM:** Evaluar a futuro la actualización del equipo a 24 GB de RAM para ofrecer mayor margen al sistema operativo mientras corren simultáneamente Ollama, ChromaDB, la base de datos y los entornos de programación.